SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management.

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	Rakshana A.R	Register Number	1921260027
Department / Section	B.Tech IT	Academic Year / Semester	1st year
Faculty Name	Dr.Arul Raj A.M	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date		Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Existing Data Centre and Requirements	3		Definition; characteristics; deployment models; scenario mapping
2	Virtualization Fundamentals and Hypervisors	3		Timeline; hardware evolution; relevance to university
3	Server Consolidation and Resource Calculations	3		Web/software evolution; SOA; APIs; scenario relevance
4	Virtual Machines and Containers	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Virtualized Storage and Network Infrastructure	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Resource Allocation, Scheduling and Monitoring	5		Responsibility layers; five-application mapping; comparison; providers
7	Availability, Security and Disaster Recovery	3		Four or more variants; two justified choices; provider risks
8	Virtualized Cloud Data Centre Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Validation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Cost, Performance and Final Recommendation	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation;

				evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations		Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution		Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices		Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification		Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots		Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap		Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation		Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code		Flask REST API and Dockerfile provided in the proposed implementation
Virtualization		Docker container configuration documented; running-container evidence required
PaaS or SaaS		Managed web-service deployment procedure described; live deployment not verified
Functional test		Expected HTTP 200 response and JSON output documented; actual output required
Failure case		HTTP 404 test case documented; actual execution record required
Screenshots		Screenshot requirements identified; authentic screenshots need verification
Reproducibility		Build commands, run commands, endpoints and expected results specified

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping		All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance		Well explained with relevant examples
3	Internet software evolution, SOA and APIs		Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations		Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs		API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping		All five applications categorized with justification
7	XaaS variants and two justified selections		Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery		Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots		Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap		Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation		Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code		Python Flask REST API and Dockerfile documented
Virtualization		Docker build and run procedures provided; execution screenshot required
PaaS or SaaS		Cloud deployment steps described; actual deployment confirmation required
Functional test		HTTP 200 and expected JSON response documented; actual test output required
Failure case		HTTP 404 test case included; execution evidence required
Screenshots		Required screenshots identified but authenticity and completeness not verified
Reproducibility		Installation, configuration, commands and testing steps documented
Interpretation		Technical outcomes and implementation limitations explained
Code and report consistency		Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	Status
All 11 tasks answered	☐ Yes ☐ Partly ☐ No
☐ Existing data-centre requirements and limitations are identified.	☐Yes ☐ Partly ☐ No
☐ Virtualization and hypervisor types are explained.	☐Yes ☐ Partly ☐ No
☐ Server consolidation calculations include working and assumptions.	☐Yes ☐ Partly ☐ No
☐ VM and container comparison and workload mapping are included.	☐Yes ☐ Partly ☐ No
☐ Storage and network virtualization are addressed.	☐ Yes ☐Partly ☐ No
☐ Resource allocation and monitoring policies are included.	☐Yes ☐ Partly ☐ No
☐ Security risks, backup and disaster recovery are discussed.	☐Yes ☐ Partly ☐ No
☐ A labelled virtualized data-centre architecture diagram is included.	☐ Yes ☐Partly ☐ No
☐ Implementation screenshots and actual test evidence are provided.	☐ Yes ☐ Partly ☐ No
☐ Three-year cost analysis and migration roadmap are included.	☐ Yes ☐ Partly ☐ No

GENERAL INSTRUCTIONS

- Use clear headings and answer each task in the order given.
- Include diagrams, tables, calculations and screenshots wherever requested.
- State assumptions and units for all numerical calculations.
- Use credible technical references and cite sources consistently.
- Ensure that implementation evidence reflects work actually performed by the team.
- Submit the report as a well-formatted document with a title page, contents, references and appendices where required.

OVERALL REMARKS

_____ **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

CSA15 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title:

Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management

(Total Marks – 50) Duration: 1 week

Scenario – University Data Centre Virtualization

A growing university operates an on-premises data centre that hosts its Student Information System, Learning Management System, Library Management System, Examination Portal, research applications and institutional website. Each major application runs on a separate physical server. Server resources are underutilized during normal periods, while admission, examination and result-publication periods create high demand. The university experiences rising electricity and maintenance costs, slow hardware procurement, limited scalability and inadequate disaster-recovery arrangements.

The current data-centre environment is assumed to have the following characteristics:

- 20 physical servers, each with 16 CPU cores, 64 GB RAM and 2 TB local storage.
- Average CPU utilisation is approximately 20%; peak utilisation reaches 90–95% during admission and examination periods.
- Most servers are dedicated to individual applications and cannot share unused capacity efficiently.
- There is no centralized virtual-machine (VM) management platform and no tested live-migration process.
- Backup is performed periodically, but a separate disaster-recovery site and documented recovery objectives are not available.
- The university wants to reduce infrastructure costs while improving availability, scalability, security and resource utilization.

You are appointed as a cloud infrastructure consultant. Analyse the existing environment and design a virtualized cloud data-centre solution. Your proposal must explain the role of hypervisors, virtual machines, containers, storage and networking virtualization, resource allocation, monitoring, security and disaster recovery. Justify all major decisions using calculations, diagrams and practical evidence.

Design Challenge:

On Day 4, the university announces that it will introduce a new digital learning platform and extend selected services to 10 affiliated colleges. The expected peak workload will increase to five times the current peak. The university requires at least 99.9% service availability, stronger tenant isolation and controlled operating costs. Re-evaluate your original design and decide whether it should be retained, modified or redesigned..

TASK 1 – Analyse the Existing Data Centre and Identify Requirements (3 Marks)

Study the current data-centre environment and identify the reasons for adopting virtualization.

Explain the purpose of a data centre and identify its major components: compute, storage, networking, power, cooling and management.

Identify at least four limitations of the existing physical-server environment.

Define resource utilization, server consolidation, scalability, availability and fault tolerance.
Prepare a requirement table covering performance, capacity, security, availability and cost.
Evidence Expected: A concise assessment of the existing environment, clearly linking each identified problem to a virtualization or cloud-infrastructure requirement.

TASK 2 – Explain Virtualization Fundamentals and Hypervisors (4 Marks)

Explain the technologies that enable multiple computing environments to share physical infrastructure.
Define virtualization and explain its purpose in a cloud data centre.
Explain the role and working of a hypervisor.
Differentiate Type-1 (bare-metal) and Type-2 (hosted) hypervisors with suitable examples such as VMware ESXi, Microsoft Hyper-V, KVM and Oracle VirtualBox.
Compare full virtualization, para-virtualization, hardware-assisted virtualization and OS-level virtualization (containers).
Recommend a suitable hypervisor for the university and justify the selection.
Evidence Expected: A labelled virtualization architecture diagram, comparison table and justified hypervisor recommendation.

TASK 3 – Design Server Consolidation and Calculate Resource Requirements (5 Marks)

Propose a consolidation plan to reduce the number of physical servers while maintaining sufficient capacity.
Explain physical-to-virtual (P2V) migration and server consolidation.
Estimate the total CPU and memory demand using the stated server specifications and average utilization. State all assumptions.
Assume a target average CPU utilization of 65% on the consolidated hosts.
Calculate the minimum number of hosts required based on CPU capacity, then check whether memory capacity changes the result.
Include a reasonable capacity reserve for peak demand and host failure; show the calculation clearly.
Discuss resource overcommitment, CPU scheduling, memory ballooning and the noisy-neighbour problem.
Evidence Expected: Worked calculations showing CPU and memory requirements, assumptions, capacity reserve and the recommended number of physical hosts. A final number without working will not receive full credit.

TASK 4 – Compare Virtual Machines and Containers (4 Marks)

Determine which workloads should run in virtual machines and which can run in containers.
Explain the architecture and characteristics of virtual machines and containers.
Compare VM and container isolation, operating-system requirements, startup time, resource overhead, portability and management.
Map at least five university workloads (for example, database, legacy student system, web portal, learning platform and analytics service) to VMs or containers.
Explain how Docker and a container orchestration platform such as Kubernetes can support deployment and scaling.

Identify situations where a VM is more appropriate than a container.
Evidence Expected: A VM-versus-container comparison and a workload mapping table supported by technical justification.

TASK 5 – Design Virtualized Storage and Network Infrastructure (4 Marks)

Design storage and network resources that support the proposed virtualized data centre.
Explain storage virtualization and compare direct-attached storage (DAS), network-attached storage (NAS) and storage area networks (SAN).
Explain software-defined storage (SDS), thin provisioning, snapshots, replication and backup.
Explain network virtualization, virtual switches, VLANs, virtual networks and software-defined networking (SDN).
Propose separate logical network segments for administration, student services, databases and management traffic.
Recommend suitable storage and network approaches for performance, security and availability.
Evidence Expected: A labelled storage and network design with explanations of the selected technologies and their roles.

TASK 6 – Plan Resource Allocation, Scheduling and Monitoring (5 Marks)

Develop a resource-management strategy that allocates infrastructure capacity efficiently.
Explain CPU, memory, storage and network resource allocation to VMs and containers.
Compare static allocation and dynamic allocation, including resource reservations, limits and shares.
Explain load balancing, workload scheduling, auto-scaling and high availability.
Identify suitable performance indicators: CPU utilization, memory utilization, disk I/O, network throughput, latency and VM availability.
Propose monitoring and alerting methods, and explain how the administrator should respond to resource contention.
Evidence Expected: A resource-allocation policy table and a monitoring plan that includes measurable indicators, thresholds and corrective actions.

TASK 7 – Evaluate Data Centre Availability, Security and Disaster Recovery (4 Marks)

Identify the risks associated with a virtualized data centre and propose suitable controls.
Explain hypervisor security, VM isolation, secure VM images, patch management and privileged access control.
Discuss VM escape, VM-to-VM attacks, insecure snapshots, VM sprawl and denial-of-service risks.
Recommend network segmentation, encryption, multi-factor authentication, role-based access control and audit logging.
Explain backup, replication, snapshots, live migration and disaster recovery.
Define Recovery Time Objective (RTO) and Recovery Point Objective (RPO), and propose suitable targets for at least two university applications.

Evidence Expected: A risk-and-mitigation table and a disaster-recovery plan with justified RTO and RPO values.

TASK 8 – Create the Virtualized Cloud Data Centre Architecture (5 Marks)

Construct an end-to-end architecture integrating compute, storage, network, virtualization and management components.

Draw a labelled architecture showing users, internet or campus network, firewall, load balancer, physical hosts, hypervisor layer, VMs/containers, virtual network, storage and monitoring.

Show how the university applications are distributed across hosts and how shared resources are allocated.

Include a management plane for provisioning, monitoring, logging and policy enforcement.

Show backup or replication to a secondary site or cloud region.

Justify the architecture in terms of scalability, performance, fault tolerance, security and cost.

Evidence Expected: A complete, clearly labelled architecture diagram and a written justification for each major component and design decision.

TASK 9 – Hands-on Implementation and Validation (5 Marks)

Build a small proof-of-concept that demonstrates virtualization and resource management.

Create at least one VM using VirtualBox, VMware, Hyper-V or KVM, or create a container using Docker.

Deploy a simple university-related service, such as a web page, student-record API or database-backed application.

Configure CPU, memory and storage resources and record the selected values.

Perform functional tests and at least one resource or load test. Record the observations.

Capture screenshots of the VM/container configuration, running service, resource-monitoring output and test results.

Document installation steps, commands, configuration details, issues faced and how they were resolved.

Evidence Expected: Reproducible implementation steps, source/configuration files, annotated screenshots and actual test evidence. Do not present simulated results as actual execution.

TASK 10 – Evaluate Cost, Performance and Final Recommendation (3 Marks)

Evaluate the proposed solution against the existing physical data-centre arrangement.

Compare the current physical-server model and proposed virtualized model in terms of hardware cost, electricity, cooling, maintenance, licensing and administration effort.

Prepare a simple three-year cost comparison using clearly stated assumptions.

Separate one-time capital expenditure from recurring operational expenditure.

Evaluate expected improvements in utilization, scalability, availability and manageability.

Identify risks such as licensing costs, vendor lock-in, skills requirements, hardware failure and migration downtime.
Provide a phased migration roadmap, identifying which applications should be migrated first and why.
Evidence Expected: A three-year cost table, risk analysis, migration roadmap and evidence-based final recommendation.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Virtualized Infrastructure (8 Marks)

Changed Requirement

The university introduces a digital learning platform and extends selected services to 10 affiliated colleges. The expected peak workload increases to five times the current peak. The system must achieve 99.9% availability, provide secure isolation between colleges and remain within a limited operating budget.
Re-evaluate your original design. You may RETAIN, MODIFY or REDESIGN the proposed infrastructure.

Impact Analysis – 2 Marks

Identify changes in workload, users, storage, network traffic and availability requirements.
Explain how multi-tenant access affects VM/container isolation, network segmentation and access control.
Identify which original capacity and cost assumptions are no longer valid.

Capacity and Availability Evaluation – 2 Marks

Estimate the additional CPU, memory, storage and network capacity needed for the five-times peak workload.
Evaluate whether the proposed number of hosts and the existing failover arrangement are sufficient.
Explain the meaning of 99.9% availability (approximately 8.76 hours of downtime per year) and identify measures needed to approach it.

Adaptation or Redesign – 2 Marks

Decide whether to retain, modify or redesign the original infrastructure and justify the decision.
Describe the revised architecture. Consider clustering, live migration, auto-scaling, container orchestration, multi-zone deployment or hybrid-cloud capacity where appropriate.
Explain changes to resource-allocation policies, tenant isolation, backup and disaster recovery.

Evidence-Based Defence – 2 Marks

Support the revised design with capacity calculations and clearly stated assumptions.
Compare the cost and operational complexity of the original and revised designs.
Discuss trade-offs involving performance, availability, security, licensing and management effort.
Evidence Expected: A technically justified decision supported by revised architecture, capacity calculations, availability considerations, security controls and cost comparison.

.Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks

Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks

Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks

Task 1	Existing Data Centre and Requirements	L2 – Understand / Analyse	3
Task 2	Virtualization Fundamentals and Hypervisors	L2 – Understand	3
Task 3	Server Consolidation and Resource Calculations	L3 – Apply	3
Task 4	Virtual Machines and Containers	L4 – Analyse	5
Task 5	Virtualized Storage and Network Infrastructure	L3/L4 – Apply / Analyse	4
Task 6	Resource Allocation, Scheduling and Monitoring	L4 – Analyse	5
Task 7	Availability, Security and Disaster Recovery	L4 – Analyse	3
Task 8	Virtualized Cloud Data Centre Architecture	L6 – Create	5
Task 9	Hands-on Implementation and Validation	L3 – Apply	4
Task 10	Cost, Performance and Final Recommendation	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4/L5/L6 – Analyse / Evaluate / Create	8
	TOTAL		50

DESIGN, ANALYSE AND VALIDATE A VIRTUALIZED CLOUD INFRASTRUCTURE

1. Introduction:

A data centre is a centralized facility that contains computing, storage, networking and supporting infrastructure required to store, process, manage and deliver digital services. Universities depend on data centres to operate essential systems such as Student Information Systems (SIS), Learning Management Systems (LMS), Library Management Systems, examination portals, research applications and institutional websites.

The university in the given scenario operates 20 physical servers. Each server has 16 CPU cores, 64 GB RAM and 2 TB of local storage. Although average CPU utilization is approximately 20%, demand rises to 90–95% during admission and examination periods. This difference between normal and peak demand indicates that resources are not being used efficiently throughout the year.

The existing arrangement also involves separate physical servers for major applications, increasing electricity consumption, cooling requirements and maintenance effort. The absence of centralized virtual-machine management and a tested live-migration process limits flexibility. Periodic backups without a separate disaster-recovery site create additional operational risk.

This assignment analyses these limitations and develops the conceptual foundation for a virtualized cloud data-centre solution. The proposed approach aims to improve resource utilization, scalability, availability, security and manageability while controlling infrastructure costs.

1.1 Objectives:

The main objectives of this assignment are:

1. To analyse the existing university data-centre environment and identify its operational limitations.
2. To explain virtualization, hypervisors and the technologies that enable shared physical infrastructure.
3. To establish the basis for consolidating physical servers into a smaller, efficiently managed host environment.
4. To relate each proposed technology to the university's application, performance, security and availability requirements.

1.2 Existing University Data-Centre Scenario:

The existing infrastructure is summarized below.

Parameter	Existing Condition
Number of physical servers	20
CPU per server	16 cores
Memory per server	64 GB RAM
Local storage per server	2 TB
Average CPU utilization	Approximately 20%
Peak CPU utilization	Approximately 90–95%
Application arrangement	Major applications run on separate physical servers
Management	No centralized VM management platform
Migration capability	No tested live-migration process
Backup and recovery	Periodic backup without a separate DR site or documented recovery objectives

TASK 1 – ANALYSE THE EXISTING DATA CENTRE AND IDENTIFY REQUIREMENTS

1.1 Introduction:

A data centre is a facility that contains computer servers, storage devices, networking equipment and other systems. It is used to store, process and manage data.

The university data centre supports different applications such as:

- Student Information System (SIS)
- Learning Management System (LMS)

- Library Management System
- Examination Portal
- Research Applications
- University Website

1.2 Major Components of a Data Centre:

Component	Purpose	University example / requirement
Compute	Executes applications and databases through CPUs, memory and servers.	Hosts SIS, LMS, examination portal and analytics.
Storage	Persists operating systems, databases, files, backups and logs.	Student records, learning materials, research datasets and archived results.
Networking	Connects users, servers, storage and external networks.	Campus LAN, internet edge, VLANs, routing, firewalls and load balancers.
Power	Supplies stable electricity and backup power.	UPS, redundant power distribution and generator support.
Cooling	Maintains safe operating temperature and humidity.	Precision cooling, airflow management and temperature sensors.
Management	Monitors, configures, secures and maintains the environment.	Hypervisor console, inventory, logging, alerting, patching and automation.

1.3 Limitations of the Existing Data Centre:

Low Resource Utilization: Most servers use only 20% of their CPU capacity during normal periods.

High Cost: Maintaining 20 separate physical servers increases electricity and maintenance costs.

Limited Scalability: Increasing server capacity requires purchasing new hardware.

Poor Resource Sharing: Unused resources from one server cannot easily be shared with another.

No Centralized Management: There is no centralized virtual machine management system.

Disaster Recovery Issues: There is no separate disaster recovery site.

1.4 Important Data-Centre Terms:

The following terms are important for understanding and designing an efficient virtualized data centre.

Term	Definition
Resource utilization	The proportion of a resource's available capacity that is actively used during a period; for example, CPU utilization = used CPU capacity ÷ total available CPU capacity × 100%.
Server consolidation	Combining workloads previously spread across multiple physical servers onto fewer, more capable hosts while preserving performance and isolation.
Scalability	The ability of a system to handle increased workload by adding resources vertically (larger resources) or horizontally (more instances).
Availability	The proportion of time a service is operational and accessible. It is commonly expressed as a percentage over a stated period.
Fault tolerance	The ability to continue operating, possibly with reduced capacity, when a component fails through redundancy and failover.

1.5 University Requirements:

	Requirement	Proposed acceptance measure
Performance	Maintain acceptable response time during admissions and examinations.	Define service-specific latency targets; test at forecast peak and 1.5× peak.
Capacity	Pool CPU, RAM, storage and network capacity; reserve failover capacity.	Normal average host CPU ≤65%; N+1 capacity available; monthly capacity review.
Security	Protect student, staff, examination and research information.	MFA for administrators; RBAC; encryption; network segmentation; audit logs.
Availability	Reduce single points of failure and recover services predictably.	Clustered hosts, redundant network/storage paths, documented RTO/RPO and DR tests.
Cost	Reduce idle hardware and operating expense without over-provisioning.	Three-year TCO comparison; track cost per application/service and cloud spend.

Conclusion: Virtualization helps the university reduce hardware costs, improve resource utilization and provide better performance and availability.

TASK 2 – EXPLAIN VIRTUALIZATION FUNDAMENTALS AND HYPERVISORS

2.1 Introduction:

Modern cloud data centres must support multiple applications while maintaining efficient resource utilisation, security, scalability, and availability. Traditionally, organisations used separate physical servers for individual applications. However, this approach often resulted in underutilised hardware, increased maintenance costs, and difficulty managing resources.

Virtualization addresses these limitations by allowing multiple independent computing environments to operate on a single physical server. It is one of the fundamental technologies behind cloud computing because it enables flexible resource allocation, server consolidation, and efficient infrastructure management.

For the university data centre, virtualization can help consolidate its 20 physical servers and provide a more efficient platform for hosting applications such as the Student Information System (SIS), Learning Management System (LMS), Library Management System, and Examination Portal.

2.2 Virtualization:

Virtualization is a technology that creates virtual versions of physical computer resources.
It allows multiple virtual machines to run on a single physical server.

Purpose of Virtualization:

- Reduces hardware costs.
- Improves resource utilization.
- Supports multiple operating systems.
- Makes resource management easier.
- Improves flexibility and scalability.

2.3 Hypervisor:

A hypervisor is software that creates and manages virtual machines.
It allows multiple virtual machines to share the resources of one physical computer.

Functions of a Hypervisor:

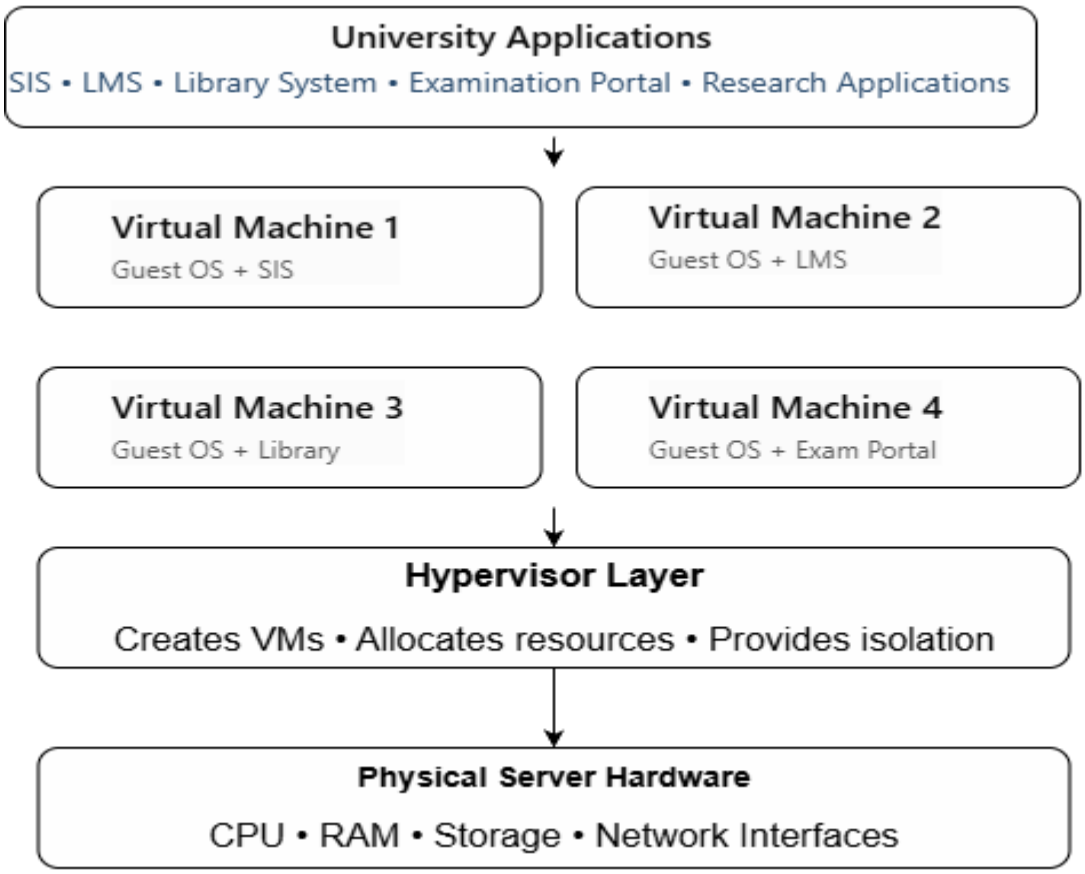
- Creates virtual machines.
- Allocates CPU and memory.
- Manages storage resources.
- Provides isolation between virtual machines.
- Controls hardware resource sharing.

Working of a hypervisor:

The hypervisor works through the following sequence:

1. The physical server provides CPU, memory, storage, and network resources.
2. The hypervisor is installed on or above the physical hardware.
3. The administrator creates virtual machines and assigns resources to them.
4. Each virtual machine runs its own guest operating system.
5. Applications operate inside their respective virtual machines.
6. The hypervisor manages and distributes physical resources among the virtual machines.

Labelled virtualization architecture diagram



2.3 Types of Hypervisors:

Hypervisors are broadly classified into two types based on how they interact with the physical hardware.

A. Type 1 Hypervisor (Bare-Metal)

A Type 1 hypervisor runs directly on the physical server hardware without requiring a conventional host operating system beneath it.

Characteristics:

- Direct access to physical hardware.
- Suitable for enterprise and cloud data centres.
- Supports centralized virtual machine management through compatible management platforms.
- Designed for production workloads.
- Provides features such as resource allocation, isolation, and migration, depending on the platform.

Examples:

- VMware ESXi
- Microsoft Hyper-V
- KVM (Kernel-based Virtual Machine)

B. Type 2 Hypervisor (Hosted)

A Type 2 hypervisor runs as an application on top of an existing host operating system, such as Windows or Linux.

Characteristics:

- Requires a host operating system.
- Easier to install and use for learning and testing.
- Depends on the host operating system for hardware access.
- Commonly used for development, demonstrations, and laboratory environments.

Example:

- Oracle VirtualBox
- VMware Workstation

Comparison between Type 1 and Type 2 hypervisors

Feature	Type-1 (bare-metal)	Type-2 (hosted)
Installation	Runs directly on physical server hardware.	Runs as an application on a host operating system.
Performance	Usually lower overhead and suitable for production server workloads.	Additional host-OS layer can add overhead.
Management	Designed for centralized data-centre management and clustering.	Often managed locally; convenient for desktop/lab use.
Examples	VMware ESXi, Microsoft Hyper-V, KVM-based platforms.	Oracle VirtualBox, VMware Workstation.
University suitability	Recommended for production cluster.	Useful for student proof-of-concept and testing.

2.6 Comparison of Virtualization Techniques

Virtualization can be implemented through different techniques depending on how the hardware and operating system resources are abstracted.

Technique	How it works	Strength / limitation
Full virtualization	Hypervisor presents virtual hardware so an unmodified guest OS can run.	Broad compatibility; some virtualization overhead.
Para-virtualization	Guest OS is aware of virtualization and uses hypercalls or optimized drivers.	Can improve efficiency; may require guest support or special drivers.
Hardware-assisted virtualization	CPU features such as Intel VT-x/AMD-V support guest execution and privilege handling.	Improves performance and simplifies running unmodified guest OSs; depends on CPU support.
OS-level virtualization	Containers isolate processes while sharing the host OS kernel.	Lightweight and fast; does not provide a separate kernel per container.

TASK 3 – DESIGN SERVER CONSOLIDATION AND CALCULATE RESOURCE REQUIREMENTS

3.1 Introduction:

Server consolidation means combining multiple physical servers into fewer powerful servers.
It helps reduce hardware costs and improves resource utilization.
The university currently has 20 physical servers.
Each server contains:

- 16 CPU cores
- 64 GB RAM
- 2 TB storage

3.2 Total Resource Calculation:

Total CPU:
20 × 16 = 320 CPU cores.
Total RAM:
20 × 64 = 1280 GB.
Total Storage:
20 × 2 = 40 TB.

3.3 Average CPU Demand:

Average CPU utilization = 20%.
320 × 20 / 100 = 64 CPU cores.
Therefore, the average CPU demand is **64 cores**.

3.4 Peak CPU Demand:

Peak CPU utilization = 90–95%.
At 90%:
320 × 90 / 100 = 288 cores.

At 95%:
320 × 95 / 100 = 304 cores.
Therefore, peak CPU demand is between **288 and 304 cores**.

3.5 Memory Requirement:

Assumption: Average memory utilization is 60%, because the question does not provide a memory utilization value.
1280 × 60 / 100 = 768 GB.
Therefore, average memory demand is **768 GB**.

3.6 Host Requirement Calculation:

Assume each new physical host contains:

- 48 CPU cores
- 512 GB RAM

Target average CPU utilization = 65%.
Usable CPU per host:
48 × 65 / 100 = 31.2 cores.
Required hosts:
64 / 31.2 = 2.05.
Rounding up, minimum required hosts = **3**.
Memory requirement:
768 / 512 = 1.5.
Therefore, minimum memory requirement = 2 hosts.
CPU determines the minimum average-load requirement.

3.7 Peak Load and Failure Calculation:

Peak CPU demand = 304 cores.
Required active hosts:
304 / 48 = 6.33.
Rounding up = 7 hosts.
To support one host failure, one additional host is required.
7 + 1 = **8 hosts**.
After one host failure:
7 × 48 = 336 CPU cores.
Therefore, 336 cores are available, which is sufficient for the estimated 304-core peak.

3.8 Resource Management Techniques:

- **CPU Overcommitment:** Allocating more virtual CPU resources than available physical cores.
- **CPU Scheduling:** Distributing CPU processing time among virtual machines.
- **Memory Ballooning:** Reclaiming unused memory from virtual machines.
- **Noisy Neighbour Problem:** One virtual machine using excessive resources and affecting other virtual machines.

3.9 Physical-to-Virtual Migration:

Physical-to-Virtual (P2V) migration is the process of converting a physical server into a virtual machine.
Steps:

1. Identify the physical server.
2. Take a backup.
3. Convert the physical server into a virtual machine.
4. Move it to the virtualization platform.
5. Test the application.
6. Start using the virtual machine.

Conclusion: The recommended initial design is 8 physical hosts with 48 CPU cores and 512 GB RAM each. This provides capacity for the current peak workload and one host failure, subject to performance testing.

TASK 4 – COMPARE VIRTUAL MACHINES AND CONTAINERS

4.1 Virtual Machines:

A Virtual Machine (VM) is a software-based computer that runs its own operating system.
Each VM has virtual CPU, memory, storage and network resources.

Advantages:

- Strong isolation.
- Supports different operating systems.
- Suitable for legacy applications.
- Provides independent environments.

4.2 Containers:

Containers are lightweight environments used to run applications.
They share the host operating system kernel but have separate application environments.
Advantages:

- Fast startup.
- Less resource usage.
- Easy application deployment.
- Supports scalability.
- Portable between compatible environments.

4.3 Comparison Between VM and Containers:

Dimension	Virtual machine	Container
Isolation	Hardware-virtualization boundary; separate guest OS and kernel.	Process and namespace/cgroup isolation; shared host kernel.
Operating system	Can run a different supported guest OS from the host.	Must use a compatible kernel family with the host.
Startup	Usually slower because a full OS boots.	Usually seconds or less because no separate OS boots.
Overhead	Higher due to guest OS and virtual hardware.	Lower; shares host kernel and uses layered images.
Portability	Portable as VM images, subject to hypervisor/format compatibility.	Portable as images across compatible container runtimes.
Management	Hypervisor and guest OS patching, VM templates and snapshots.	Image registry, orchestration, deployment manifests and runtime security.
Best use	Databases, legacy systems, different OS requirements, sensitive isolation.	Stateless web services, APIs, workers and independently scalable services.

4.4 University Workload Mapping:

University workload	Recommended platform	Reason
Student Information System (legacy)	VM	Legacy dependencies and possible OS-specific requirements; migrate carefully.
Relational database	Dedicated VM initially; managed DB later if justified	Predictable storage I/O, memory reservation, backup and controlled maintenance.
Public web portal	Containers	Stateless web tier can be replicated and updated independently.
Learning Management System	VM for monolithic core; containers for separable services	Supports compatibility while allowing new components to scale horizontally.
Examination portal/API	Containers behind load balancer and API gateway	Rapid scale-out during examination windows; immutable deployments.
Research analytics service	Container jobs or dedicated VMs depending on libraries/GPU needs	Batch workers can scale by queue depth; special hardware may favour VMs.
Library Management System	VM initially	Preserves existing application stack while allowing consolidation.

4.5 Role of Docker and Kubernetes:

Docker:

Docker is a platform used to create and run containers.

It packages applications with their required libraries and dependencies.

Kubernetes:

Kubernetes is a container orchestration platform.

Its main functions are:

- Manages multiple containers.
- Automatically restarts failed containers.
- Supports load balancing.
- Provides automatic scaling.
- Helps in application deployment.

4.6 When to Use Virtual Machines:

Virtual machines are more suitable when:

- An application requires a different operating system.
- Strong isolation is needed.
- Legacy applications are used.
- Full operating system control is required.

Conclusion: Virtual machines are suitable for databases and legacy applications, while containers are suitable for websites, APIs and modern applications.

TASK 5 – DESIGN VIRTUALIZED STORAGE AND NETWORK INFRASTRUCTURE

5.1 Introduction:

A virtualized cloud data centre requires an efficient storage and networking infrastructure to support multiple virtual machines, containers, and university applications.

Storage virtualization allows multiple physical storage devices to be combined and presented as logical storage resources. Network virtualization creates logical networks over shared physical networking infrastructure.

For the university, these technologies can provide centralized storage, improved resource utilization, secure communication, and better availability for applications such as the Student Information System (SIS), Learning Management System (LMS), Library Management System, Examination Portal, and research applications.

5.2 Storage Virtualization:

Storage virtualization is the process of combining storage capacity from multiple physical storage devices and presenting it as a single logical storage resource.

It separates the logical view of storage from the actual physical storage devices. Administrators can allocate, manage, and monitor storage without requiring applications to know the exact physical location of their data.

Working of storage virtualization

1. Multiple physical storage devices are connected to the storage infrastructure.
2. A storage virtualization layer combines or manages these storage resources.
3. The available capacity is presented as logical storage volumes or shared storage.
4. Virtual machines and applications access the allocated storage.
5. Administrators monitor storage usage and allocate additional capacity when required.

Benefits of storage virtualization

- Centralized management: Storage resources can be managed from a common platform.
- Improved utilization: Unused storage capacity can be allocated to other applications.
- Scalability: Additional storage can be added when university data grows.
- Data availability: Replication and redundant storage configurations can improve availability.
- Simplified administration: Storage allocation and monitoring become easier.
- Flexible allocation: Different applications can receive storage according to their requirements.

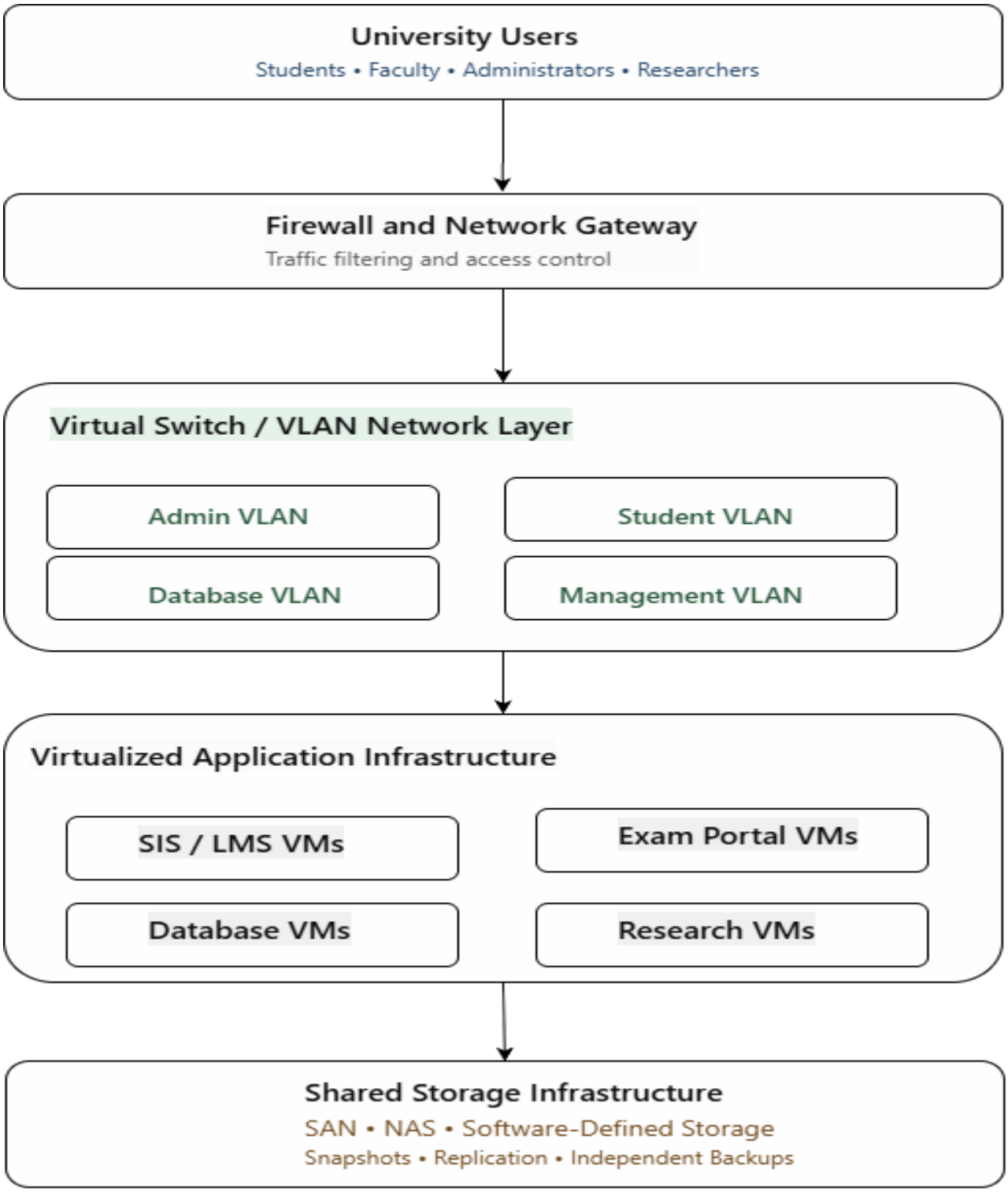
5.3 Network Virtualization:

Network virtualization creates logical networks over shared physical switches and links. A virtual switch connects VMs and containers on a host. VLANs separate Layer-2 broadcast domains; virtual

networks and overlays can extend logical segmentation across hosts. Software-defined networking (SDN) centralizes policy and network programming through controllers and APIs.

Segment / VLAN	Example purpose	Access policy
VLAN 10 – Administration	Staff administration and approved management workstations.	Allow only authorized staff; MFA and least privilege.
VLAN 20 – Student services	Student portal, LMS front end and public-facing services.	Permit required HTTPS access; block direct database access.
VLAN 30 – Database	SIS, LMS and examination databases.	Only approved application servers may connect on required ports.
VLAN 40 – Management	Hypervisor management, storage management and monitoring.	Restricted to management jump hosts; never exposed directly to internet.
VLAN 50 – Backup / replication	Backup traffic and inter-site replication.	Dedicated ACLs, encryption and bandwidth controls.
VLAN 60 – Research / tenant workloads	Research applications and affiliated-college workloads.	Tenant-specific policies and quotas; deny cross-tenant traffic by default.

5.4Labelled storage and network design:



5.5 Recommended Storage and Network Design:

- Use shared storage for virtual machines.
- Use redundant network switches.
- Use VLANs to separate different departments.
- Use encryption to protect data.
- Use backup and replication for data protection.
- Use SDN for centralized network management.

Conclusion: Virtualized storage and networking improve data security, resource sharing, performance and availability.

TASK 6 – PLAN RESOURCE ALLOCATION, SCHEDULING AND MONITORING

6.1 Resource Allocation:

Resource allocation is the process of assigning CPU, memory, storage and network capacity to virtual machines (VMs) and containers according to application demand. The university should use a centrally managed resource pool so that capacity can be shared while critical services retain guaranteed resources.

6.2 Allocation of Infrastructure Resources:

Resource	Allocation method	University application / policy
CPU	Assign virtual CPUs (vCPUs), CPU reservations, limits and shares. Use scheduling to distribute physical CPU time.	Reserve sufficient CPU for SIS and the Examination Portal. Allow LMS and web services to scale during admission and examination periods.
Memory	Assign VM/container memory based on measured working-set demand. Use reservations for critical workloads and avoid unsafe overcommitment.	Protect database memory and keep host-level memory headroom. Monitor swapping and ballooning.
Storage	Allocate virtual disks or persistent volumes with capacity quotas, IOPS/throughput policies and growth alerts.	Use high-performance storage for databases and examination records; use scalable shared storage for learning materials and research files.
Network	Assign virtual NICs, bandwidth policies, VLANs/overlays and traffic priorities.	Prioritise examination and SIS traffic; isolate management, database, student and research networks.

6.3 Static and Dynamic Resource Allocation:

Static allocation assigns a predefined amount of resources to a VM or application. It is predictable and easy to control, but unused capacity may remain idle. Dynamic allocation adjusts resources according to measured demand and configured policies. It improves utilization and responsiveness, but requires monitoring and safeguards.

Method	Meaning	Advantages	Limitations / suitable use
Static allocation	CPU, memory or storage capacity is assigned in advance and remains fixed unless an administrator changes it.	Predictable performance; simple capacity planning; useful for critical databases.	May waste capacity during quiet periods; less flexible during sudden peaks.
Dynamic allocation	Resources are adjusted automatically or manually as workload changes.	Better utilization; supports variable LMS, portal and analytics demand.	Needs thresholds, monitoring and limits to prevent uncontrolled growth.

Three important hypervisor controls are:

- Reservation: the minimum amount of a resource guaranteed to a VM.
- Limit: the maximum amount of a resource that a VM can consume.
- Shares: relative priority when multiple VMs compete for a resource; shares do not guarantee a fixed quantity.

6.4 Proposed Resource Allocation Policy:

University workload	Suggested allocation policy	Priority and reason
Student Information System (SIS)	Dedicated VM; reserved CPU and memory; database storage with high IOPS; controlled network access.	Critical. Student records and registration require predictable response and data integrity.
Learning Management System (LMS)	Multiple web/application instances; dynamic CPU scaling; shared persistent storage.	High. Demand varies with class schedules and online submissions.
Examination Portal	Isolated application instances; CPU and memory reservations during examinations; rate limits and priority network policy.	Highest during examinations. Protects assessment availability and prevents noisy-neighbour effects.
Library Management System	VM initially; moderate reservations; scheduled backup and storage monitoring.	Medium. Workload is comparatively steady, but catalogue and circulation data must be protected.
Research and analytics	Container workers or dedicated VMs; quotas and scheduled/batch queues; separate tenant/network segment.	Variable. Workloads can be scheduled outside critical academic periods and must not starve core services.

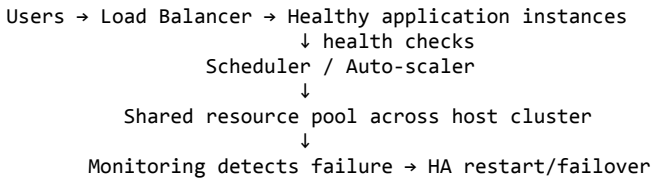
6.5 Load Balancing, Scheduling, Auto-Scaling and High Availability:

Load balancing distributes incoming requests across multiple healthy application instances. A load balancer checks instance health and sends new requests only to available instances. This prevents one server from receiving all traffic and improves response time.

Workload scheduling decides where and when VMs, containers or batch jobs should run. A scheduler considers available CPU, memory, storage, network capacity, application priority, affinity rules and maintenance status. Examination jobs receive priority during scheduled examination windows, while non-urgent analytics jobs can be delayed.

Auto-scaling changes the number or size of application instances in response to demand. Horizontal scaling adds or removes instances; vertical scaling increases or decreases the resources assigned to an instance. Scale-out is preferred for stateless web and API services, while database scaling requires controlled capacity planning.

High availability (HA) uses clustered hosts, redundant network and storage paths, health monitoring and automatic restart/failover. A failed VM may restart on another host, provided sufficient spare capacity and shared or replicated storage are available. HA reduces service interruption but does not replace backups or disaster recovery.



6.6 Performance Indicators and Monitoring Plan:

Indicator	What it measures	Proposed warning / critical threshold	Administrator response
CPU utilization	Percentage of allocated CPU capacity in use.	Warning: >75% for 10 minutes. Critical: >90% for 5 minutes.	Identify top consumers; rebalance or scale out; review CPU limits and host capacity.
Memory utilization	Used memory compared with available memory.	Warning: >80%. Critical: >90% or sustained swapping.	Check memory leaks and ballooning; add memory or move workloads; reduce unsafe overcommitment.
Disk I/O latency	Time required to complete storage read/write operations.	Warning: >20 ms. Critical: >50 ms for sustained periods.	Inspect storage queue depth and IOPS; move database volumes to faster storage; check backup contention.
Network throughput	Traffic compared with available link capacity.	Warning: >80%. Critical: >90% sustained.	Check traffic sources; apply QoS; add bandwidth or distribute traffic across redundant links.
Application latency	Time taken to respond to a user request.	Warning: >20% above the agreed service target. Critical: >50% above target.	Inspect application, database and network traces; add instances or fix bottlenecks.
VM availability	Percentage of time a VM/service is operational.	Target: at least 99.9% for designated critical services; alert on any unplanned outage or repeated restart.	Trigger incident procedure; fail over/restart; investigate root cause and record downtime.

These thresholds are proposed starting values, not measured values supplied in the scenario. The administrator should establish a baseline, tune thresholds for each application and review them after load testing.

A monitoring platform should collect hypervisor, VM, container, storage, network and application metrics. It should provide dashboards, central logs, alert notifications, capacity trends and audit records. Alerts should be routed to the responsible administrator and escalated according to service priority.

6.7 Response to Resource Contention:

Problem	Possible cause	Corrective action
CPU contention	Too many busy VMs on one host; excessive CPU overcommitment.	Use scheduler metrics; move or scale workloads; adjust shares and limits; add host capacity.
Memory pressure	Insufficient host RAM, memory leaks or excessive VM allocation.	Find high-memory processes; right-size VMs; add RAM; avoid relying on swapping.
Storage bottleneck	High I/O workload, slow disks or backup jobs running during peak hours.	Use faster storage tiers; distribute I/O; schedule backups; monitor latency and queue depth.
Network congestion	Heavy downloads, replication or poorly isolated traffic.	Apply QoS and VLAN policies; reserve bandwidth for critical services; expand links where justified.
Noisy neighbour	One VM/container consumes disproportionate shared resources.	Apply quotas, limits and reservations; isolate the workload; review tenant usage.

Conclusion: A combination of reservations for critical applications, dynamic scaling for variable workloads, centralized monitoring and planned failover will improve resource utilization without allowing one application to affect the performance of other university services.

TASK 7 – EVALUATE DATA CENTRE AVAILABILITY, SECURITY AND DISASTER RECOVERY

7.1 Introduction:

A virtualized data centre centralizes many workloads on shared hosts, storage and networks. This improves flexibility but also means that a failure or security incident affecting a shared component can affect several applications. The university must therefore combine technical security controls, redundancy, tested backups and a documented disaster-recovery (DR) plan.

7.2 Hypervisor and Virtual Machine Security:

- Hypervisor security: install the hypervisor from a trusted source, disable unused services, restrict management interfaces and apply security patches promptly.
- VM isolation: separate workloads using virtual networks, VLANs, firewall rules and least-privilege access. Database VMs should accept connections only from approved application services.
- Secure VM images: maintain approved golden images, remove default accounts and unnecessary software, apply updates and scan images before deployment.
- Patch management: maintain an asset inventory, test patches in a staging environment, schedule maintenance windows and verify the result.
- Privileged access: use role-based access control (RBAC), multi-factor authentication (MFA), separate administrator accounts and time-limited privileged access.
- Encryption and audit: encrypt sensitive data in transit and at rest, protect encryption keys, centralize logs and review administrative actions.

7.3 Virtualization Risks and Mitigation:

Risk	Explanation / possible impact	Recommended mitigation
VM escape	A vulnerability could allow activity to cross the isolation boundary between a guest VM and its host.	Patch hypervisors and guest tools; minimize exposed services; use supported configurations and security monitoring.
VM-to-VM attack	A compromised VM may attempt to reach another VM on a shared network.	Use micro-segmentation, host firewalls, VLANs, deny-by-default rules and workload-level access controls.
Insecure snapshots	Snapshots may contain sensitive memory, credentials or outdated data and may be accessible to unauthorized users.	Restrict snapshot permissions; encrypt storage; define retention; delete obsolete snapshots securely.
VM sprawl	Unmanaged or forgotten VMs consume capacity and may remain unpatched.	Require approval, ownership tags, quotas, expiry dates and periodic inventory reviews.
Denial of service (DoS)	Excessive requests or resource consumption can exhaust CPU, memory, storage or network capacity.	Use rate limiting, load balancing, resource limits, traffic filtering and capacity alerts.
Management-plane compromise	Stolen administrator credentials may provide broad control over hosts and VMs.	MFA, RBAC, isolated management network, privileged-access review and immutable audit logs.
Host or storage failure	A physical component failure may affect multiple hosted services.	N+1 host capacity, redundant storage/network paths, replication, tested backups and failover procedures.

7.4 Backup, Replication, Snapshots and Live Migration:

Backup creates a recoverable copy of data or a system at a particular point in time. Backups should follow a defined schedule, be encrypted and include at least one copy isolated from the production environment. Restoration must be tested regularly.

Replication copies data to another storage system or site. It can reduce data loss and recovery time, but replication may also copy accidental deletion or corruption; it must therefore be combined with independent backups.

A snapshot records the state of a VM disk or system at a point in time. It is useful before a change or upgrade, but it is not a substitute for an independent backup. Snapshots should have access controls and retention limits.

Live migration moves a running VM from one compatible physical host to another with little or no planned downtime, subject to platform, network and storage requirements. It supports maintenance and load balancing, but does not by itself protect against site-wide disasters.

7.5 Recovery Time Objective and Recovery Point Objective:

Recovery Time Objective (RTO) is the maximum acceptable time to restore a service after disruption. Recovery Point Objective (RPO) is the maximum acceptable amount of data loss measured in time. A smaller RTO or RPO generally requires more investment in redundancy, replication and operational readiness.

Application	Proposed RTO	Proposed RPO	Justification

Examination Portal	30 minutes	5 minutes	Examinations are time-sensitive; recovery must be rapid and submitted answers should have minimal data loss.
Student Information System (SIS)	1 hour	15 minutes	Registration and student records are critical, but a short controlled recovery window may be acceptable.
Library Management System	4 hours	1 hour	Important service, but temporary interruption is less disruptive than an examination outage.

These are proposed service targets for planning and must be approved by university management after a business-impact analysis. Actual RTO/RPO achievement must be demonstrated through recovery tests.

7.6 Disaster-Recovery Plan:

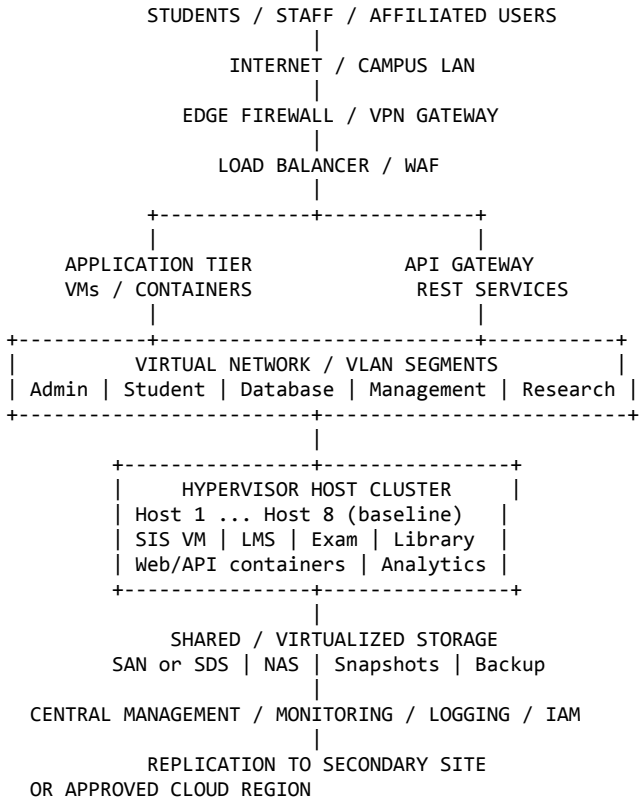
1. Identify critical services, data owners, dependencies and approved RTO/RPO values.
2. Maintain encrypted, versioned backups and replicate critical data to a separate recovery site or cloud region.
3. Keep recovery infrastructure, network routes, identity services and required software images documented.
4. Monitor replication health and backup completion; alert administrators when a job fails.
5. During a disaster, declare the incident, assess impact and activate the recovery site according to service priority.
6. Restore identity and network dependencies first, followed by databases and application services; validate data consistency.
7. Test user access and application functions before reopening the service.
8. Record recovery time, data loss, issues and corrective actions; conduct scheduled DR exercises.

Conclusion: Security must be applied at the hypervisor, VM, network, storage and administrator levels. Redundant infrastructure, independent backups and tested recovery procedures are required to protect the university's services.

TASK 8 – CREATE THE VIRTUALIZED CLOUD DATA CENTRE ARCHITECTURE

8.1 Proposed End-to-End Architecture:

The proposed architecture uses a clustered Type-1 hypervisor environment with shared storage, segmented virtual networking and centralized management. Critical university applications run on VMs, while suitable stateless web and API components run in containers. The architecture retains the eight-host baseline proposed in Task 3 for the current workload, subject to the capacity limitations discussed in Task 11.



8.2 Major Components and Justification:

Component	Role in the architecture	Justification
Users and campus / internet network	Provides access for students, staff and authorized affiliated-college users.	Supports on-campus and remote access through controlled entry points.
Firewall, VPN and WAF	Filters network traffic, provides secure remote connectivity and protects web applications.	Reduces unauthorized access and common web threats.
Load balancer and API gateway	Distributes requests and applies API routing, authentication, throttling and monitoring policies.	Prevents overload of one application instance and protects shared services.
Type-1 hypervisor cluster	Runs and isolates VMs across physical hosts.	Enables consolidation, centralized management, HA and planned live migration.
Containers and orchestration	Runs portable stateless web/API services and scales replicas.	Supports rapid deployment and horizontal scaling for variable demand.
Virtual network and VLANs	Separates administration, student, database, management, backup and research traffic.	Limits lateral movement and controls communication between security zones.
Shared storage / SDS / NAS	Provides VM disks, database volumes, shared files and persistent application data.	Enables centralized allocation, redundancy and easier capacity expansion.
Management, monitoring and logging	Provides provisioning, inventory, metrics, alerts, audit and policy enforcement.	Improves visibility and supports operational response.
Secondary site / cloud region	Receives backups and selected replicated services/data.	Supports recovery from a major site failure.

8.3 Application Placement Across the Infrastructure:

Application	Recommended placement	Placement reason
SIS and relational database	Dedicated VMs on the host cluster; database on high-performance shared storage.	Legacy compatibility, predictable I/O and stronger control over database resources.
LMS	VM for core service; containerized web/API components where supported.	Allows gradual modernization and scaling of front-end components.
Examination Portal	Isolated application VMs/containers behind a load balancer and API gateway.	Supports demand spikes, strict access control and independent scaling.
Library Management System	VM initially, with protected database storage.	Reduces migration risk for an existing application.
Research analytics	Container workers or dedicated VMs with quotas and scheduling.	Allows batch processing without starving critical university services.

8.4 Scalability, Performance, Security and Cost:

- Scalability: add host capacity to the cluster and increase application replicas when measured demand rises.
- Performance: use reservations for critical VMs, suitable storage tiers, load balancing and QoS for important network traffic.
- Fault tolerance: use N+1 host capacity, redundant network paths, shared/replicated storage and HA restart policies.
- Security: enforce MFA, RBAC, encryption, tenant segmentation, secure images, patching and centralized audit logs.
- Cost control: consolidate suitable workloads, track resource utilization, use quotas and avoid unnecessary cloud resources.
- Disaster recovery: keep independent backups and replicate selected services/data to a separate recovery location.

Conclusion: The architecture combines VMs for workloads needing independent operating systems or stronger isolation with containers for suitable modern services. Shared infrastructure is centrally monitored and segmented, while backup and secondary-site replication support recovery.

TASK 9 – HANDS-ON IMPLEMENTATION AND VALIDATION

9.1 Proof-of-Concept Objective and Evidence Status:

A small proof-of-concept (PoC) can demonstrate a university-related service running in a virtualized or containerized environment. This section provides a reproducible Docker-based student-record API procedure, configuration and test cases. The supplied assignment file does not contain verified execution logs or authentic screenshots; therefore, the commands and outputs below are implementation instructions and expected results, not a claim that the tests have already been executed.

Evidence status: the source file identifies a Flask REST API and Dockerfile as proposed implementation artefacts, but actual running-container evidence, live deployment, API output and screenshots still require capture from the user's own environment.

9.2 Proposed PoC Environment:

Item	Proposed configuration
Host environment	Windows / Linux computer with Docker Desktop or Docker Engine installed.
Application	Python Flask REST API for sample university student records.
Container image	Python slim base image with Flask dependency.
Container port	5000 inside the container, mapped to port 5000 on the host.
Resource configuration	For a demonstration, use a modest CPU and memory limit supported by the installed Docker version; record the actual values configured.
Test tools	Browser or curl / PowerShell Invoke-RestMethod; Docker CLI and resource monitoring.

9.3 Application Source Code:

Create a project folder named university-api and save the following file as app.py:

```
from flask import Flask, jsonify

app = Flask(__name__)

students = [
    {"id": 1, "name": "Ananya", "department": "IT"},
    {"id": 2, "name": "Arun", "department": "CSE"}
]

@app.get("/")
def home():
    return jsonify({
        "service": "University Student API",
        "status": "running"
    })

@app.get("/api/students")
def get_students():
    return jsonify(students), 200

@app.get("/api/students/<int:student_id>")
def get_student(student_id):
    for student in students:
        if student["id"] == student_id:
            return jsonify(student), 200
    return jsonify({"error": "Student not found"}), 404

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

9.4 Dependency and Docker Configuration:

Create requirements.txt:

```
Flask==3.1.0
```

Create a file named Dockerfile (without a file extension):

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

9.5 Build and Run Procedure:

1. Install Docker Desktop or Docker Engine and confirm that the Docker service is running.
2. Open a terminal in the university-api project folder.
3. Build the container image using the command shown below.
4. Run the container and map host port 5000 to container port 5000.
5. Check that the container is running and note its ID, image and port mapping.
6. Open the API endpoint in a browser or send requests using curl / PowerShell.
7. Record the actual CPU and memory limits and observe resource use while sending test requests.

```
docker build -t university-student-api .
docker run -d --name university-api -p 5000:5000 --cpus=1 --memory=512m university-student-api
docker ps
docker stats university-api
```

The example uses a limit of 1 CPU and 512 MB memory. If the installed platform does not support these flags, consult its version-specific documentation and record the actual configuration used.

9.6 Functional Test Cases and Expected Results:

Test	Request / action	Expected result (not an observed result)

Service health	GET http://localhost:5000/	HTTP 200 with JSON indicating University Student API and status running.
List students	GET http://localhost:5000/api/students	HTTP 200 with a JSON array containing the two sample records.
Existing student	GET http://localhost:5000/api/students/1	HTTP 200 with the record for student ID 1.
Missing student	GET http://localhost:5000/api/students/99	HTTP 404 with {"error":"Student not found"}.
Container status	docker ps	Container appears as running with the configured port mapping.
Resource observation	docker stats university-api	Displays live CPU and memory usage; record the values actually observed.

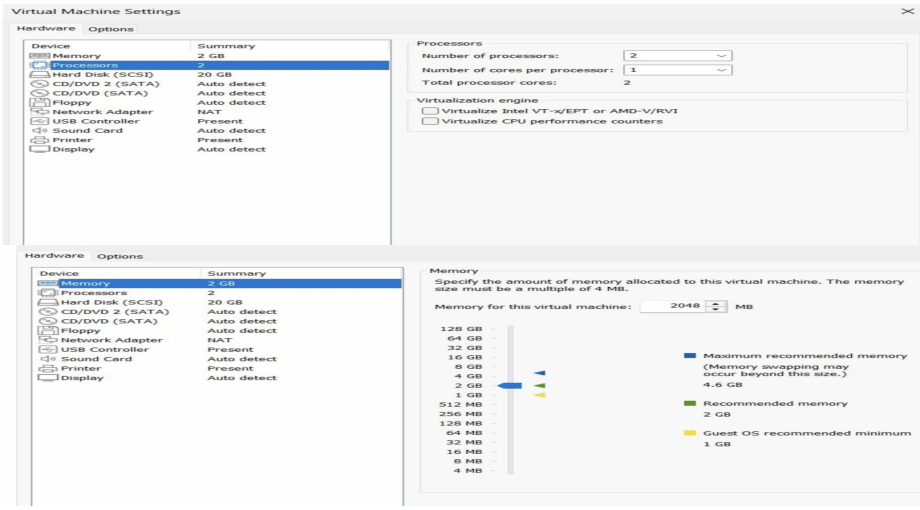
Example requests:

```
curl http://localhost:5000/
curl http://localhost:5000/api/students
curl http://localhost:5000/api/students/1
curl -i http://localhost:5000/api/students/99
```

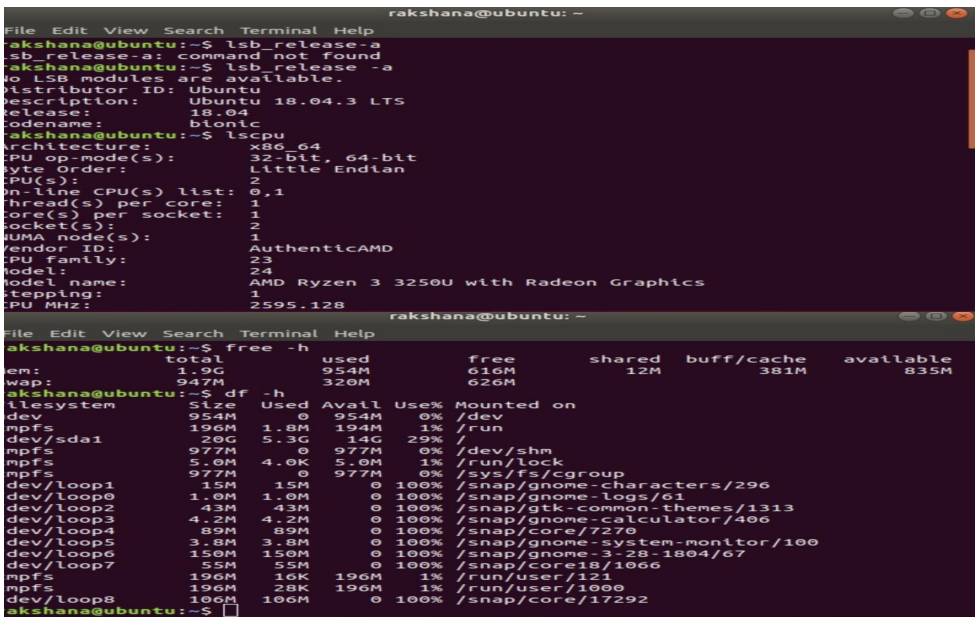
9.7 Screenshot and Execution Evidence to Attach:

Capture genuine screenshots after running the procedure. Insert each screenshot below the corresponding caption, using the same image placement and caption style as the earlier diagrams in this report.

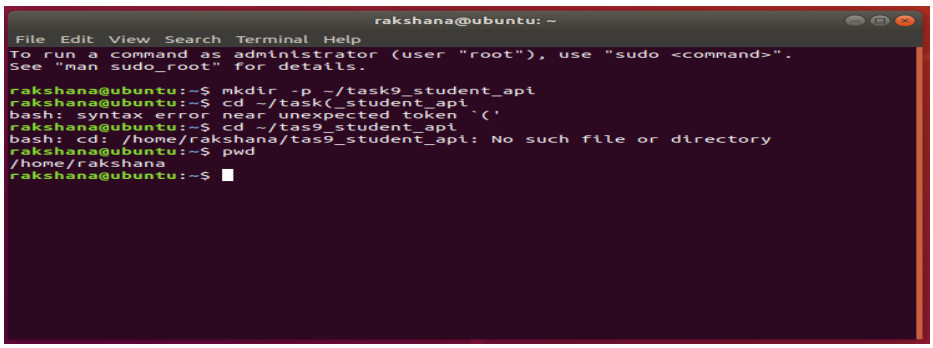
1) Practical Implementation and Testing of a Virtualized Environment:



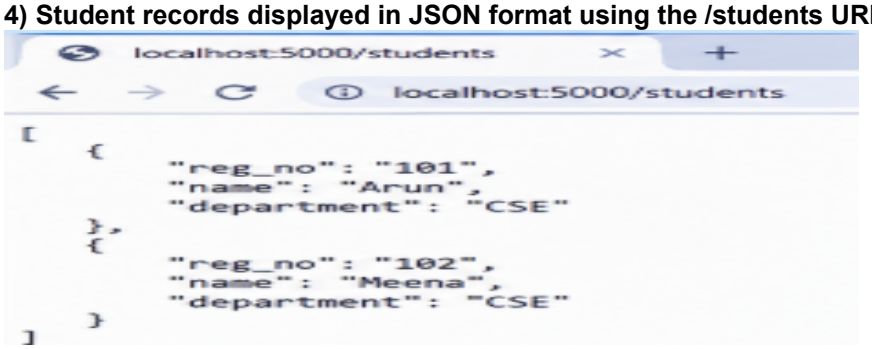
2) VMware Virtual Machine Hardware Configuration below this paragraph:



3) Flask Server Running:



4) Student records displayed in JSON format using the /students URL:



9.8 Validation and Limitations:

- Functional validation checks whether the service responds correctly to valid and invalid requests.
- Container validation checks whether the image builds, the container starts and the port is reachable.
- Resource validation records actual CPU and memory usage under a small test workload. A controlled load test should be performed only within the user's own test environment.
- The sample API uses in-memory demonstration data and is not a production student-record system. A real deployment requires authentication, authorization, input validation, HTTPS, a persistent database, secret management, backups and audit logging.
- A local Docker container demonstrates containerization; it does not by itself prove a managed PaaS/SaaS deployment or production availability.

Conclusion: The proposed PoC provides a reproducible method for packaging and testing a university-related REST API in a container. Final submission evidence must include actual commands, outputs, resource observations and authentic screenshots from the environment in which it is run.

TASK 10 – EVALUATE COST, PERFORMANCE AND FINAL RECOMMENDATION

10.1 Basis and Costing Assumptions:

The following three-year comparison is an illustrative planning estimate in Indian Rupees (₹), not a supplier quotation. Actual costs depend on server specifications, warranty, power tariffs, facility efficiency, software licensing, storage requirements and support contracts. Existing assets may also be reusable, which would reduce the actual migration cost.

Assumption	Existing physical model	Proposed virtualized model
Compute hardware	20 physical servers; assumed refresh cost ₹4.0 lakh per server.	8 virtualization hosts; assumed cost ₹8.0 lakh per host.
Shared storage / backup	Existing local storage; limited shared capacity and no separate tested DR site.	Shared storage and backup/replication infrastructure budgeted separately.
Network	Existing network; assumed ₹10 lakh refresh/upgrade allowance.	Redundant virtualized network and switching allowance.
Energy and cooling	Estimated annual facility energy cost ₹11.4 lakh.	Estimated annual facility energy cost ₹7.1 lakh.

Annual support and operations	Maintenance, software and administration estimated separately.	Support/licensing, cloud/DR services and administration estimated separately.

10.2 One-Time Capital Expenditure:

Capital item	Existing physical model (₹ lakh)	Virtualized model (₹ lakh)
Compute servers	80.0	64.0
Storage / backup / DR infrastructure	Included in existing baseline; refresh allowance not separately priced	18.0
Network upgrade	10.0	8.0
Deployment / migration / setup	—	4.0
Total one-time estimate	90.0	94.0

The virtualized option has a higher initial estimate in this simplified model because it includes shared storage, backup/DR and migration setup. These components provide capabilities that are missing or inadequate in the current environment.

10.3 Annual Operating Expenditure:

Annual cost item	Existing physical model (₹ lakh/year)	Virtualized model (₹ lakh/year)
Electricity and cooling	11.4	7.1
Hardware maintenance / support	20.0	8.0
Software / platform licensing	4.0	6.0
Backup / DR / cloud services	—	4.0
Administration effort	8.0	6.0
Total annual OPEX	43.4	31.1

10.4 Three-Year Total Cost of Ownership:

Cost measure	Existing physical model (₹ lakh)	Virtualized model (₹ lakh)
One-time capital expenditure	90.0	94.0
Three years of OPEX	43.4 × 3 = 130.2	31.1 × 3 = 93.3
Estimated three-year TCO	220.2	187.3

Estimated three-year difference = ₹220.2 lakh − ₹187.3 lakh = ₹32.9 lakh. Estimated reduction = (32.9 ÷ 220.2) × 100 = approximately 14.9%. This is an assumption-based estimate, not a guaranteed saving. Licensing, storage growth, cloud consumption and implementation costs may change the result.

10.5 Performance and Operational Comparison:

Evaluation area	Existing physical arrangement	Proposed virtualized arrangement
Resource utilization	Average CPU utilization is approximately 20%, with peaks of 90–95%; dedicated servers cannot share unused capacity easily.	Shared resource pools improve utilization, subject to reservations, limits and capacity headroom.
Scalability	Usually requires purchasing and installing another physical server.	VM resources and container replicas can be adjusted more quickly; additional hosts or cloud capacity can be added.
Availability	No centralized VM platform, tested live migration or separate DR site is stated.	Clustered hosts, redundant paths, HA restart, replication and tested DR improve resilience.
Management	Separate hardware and application administration.	Centralized provisioning, monitoring, patching, inventory and policy enforcement.
Cost control	Higher idle hardware, electricity and maintenance burden.	Potentially lower operating cost, but requires monitoring of licences, cloud usage and storage.

10.6 Risks and Controls:

Risk	Potential impact	Control
Software licensing	Virtualization or per-core licensing may increase costs as hosts or cores are added.	Confirm licensing rules before procurement; compare open-source and commercial support options.
Vendor lock-in	Proprietary VM, storage or cloud services may make migration difficult.	Prefer documented interfaces and portable formats; maintain export and exit procedures.
Skills requirements	Administrators need hypervisor, storage, network, container and security skills.	Provide training, operating procedures and vendor support during migration.
Hardware failure	A host, switch or storage failure can affect multiple workloads.	Use N+1 capacity, redundant components, monitoring and tested failover.

Migration downtime	Application conversion or data transfer may interrupt services.	Pilot low-risk workloads, back up data, use maintenance windows and prepare rollback plans.
Cloud cost growth	Uncontrolled scale-out, storage and data transfer may exceed budget.	Use budgets, quotas, tagging, alerts, scheduled scale-down and monthly cost reviews.

10.7 Phased Migration Roadmap:

Phase	Activities	Applications / reason
1. Assess and prepare	Inventory dependencies, classify data, baseline performance, design networks and backups, procure and configure hosts.	Prepare the platform before moving critical services.
2. Pilot migration	Migrate a low-risk development or test workload; validate backup, monitoring and rollback.	Confirms platform compatibility and team readiness.
3. Non-critical services	Migrate institutional website and selected research/test workloads.	Lower business impact and useful for operational learning.
4. Core application migration	Migrate Library Management System and LMS components in planned windows; test integrations.	Moderate-to-high importance; use application-specific validation.
5. Critical services	Migrate SIS, databases and Examination Portal only after performance, security and DR testing.	Highest impact; require approved downtime windows and rollback plans.
6. Optimize	Review utilization, tune reservations, test failover and DR, review cost and security.	Ensures the new environment remains stable and cost-controlled.

Final recommendation: adopt a phased move to a Type-1 hypervisor cluster with shared storage, redundant networking, centralized monitoring and a tested secondary recovery location. Use VMs for legacy and database workloads and containers for suitable stateless web/API services. Retain the existing eight-host baseline only as an initial current-workload design subject to measured capacity validation; the Day-4 demand increase requires a separate capacity and hybrid-cloud decision in Task 11.

TASK 11 – RE-EVALUATE AND ADAPT THE VIRTUALIZED INFRASTRUCTURE

11.1 Changed Requirement and Interpretation:

The main Day-4 design challenge on page 9 specifies a new digital learning platform, services extended to 10 affiliated colleges, a peak workload five times the current peak, at least 99.9% availability, stronger tenant isolation and controlled operating costs. A later repeated section on pages 10–11 refers instead to 10× load and 40 colleges, and the evidence checklist also mentions 10×/40 colleges. These statements conflict. To avoid silently changing the scenario, the primary design below uses the explicit Day-4 requirement of 5× peak and 10 colleges. A separate 10×/40-college stress case is also evaluated to address the repeated rubric wording.

11.2 Impact Analysis:

Area	Impact of the primary 5× / 10-college change	Required design response
Workload and users	More concurrent learners, staff and college users; new digital learning services; much higher peak request rates.	Scale web/API instances horizontally; load-test enrollment, streaming/content and assessment workflows.
Storage	More course content, submissions, logs, student activity and backups.	Use scalable shared/object storage, quotas, lifecycle policies and separate tenant data controls.
Network traffic	More concurrent sessions, downloads, video/content delivery, API calls and inter-site traffic.	Measure current peak throughput; provision at least 5× measured peak as the initial target, with redundancy and QoS.
Availability	The service must approach 99.9% availability, not merely operate on one host or one site.	Use clustered services, redundant network/storage, health checks, failover and a tested recovery site.
Multi-tenancy	Ten colleges share the platform but must not access one another's records or resources.	Use tenant identity, tenant-scoped authorization, network segmentation, quotas, audit logs and database-level isolation.
Cost assumptions	A small fixed cluster sized for the original workload cannot handle a sustained 5× peak economically by itself.	Use hybrid burst capacity, autoscaling, budgets, quotas and measured capacity procurement.

Web services and APIs become critical because the digital learning platform, college portals and university systems must exchange student, course, attendance, assessment and learning data. An API gateway should provide authentication, authorization, rate limiting, request validation, versioning, logging and usage monitoring.

11.3 Capacity and Availability Evaluation:

The current peak CPU demand calculated in Task 3 is 304 physical CPU cores (95% of 320 cores). For planning, retain the 48-core host assumption and the 65% target CPU utilization. This gives 48 × 0.65 = 31.2 planning CPU cores per host.

Calculation	5× peak / 10 colleges (primary scenario)	10× peak / 40 colleges (rubric stress case)

Baseline peak CPU demand	304 cores	304 cores
Scaled peak CPU demand	304 × 5 = 1,520 cores	304 × 10 = 3,040 cores
Planning capacity per 48-core host	48 × 65% = 31.2 cores	48 × 65% = 31.2 cores
Active hosts required	1,520 ÷ 31.2 = 48.72 → 49	3,040 ÷ 31.2 = 97.44 → 98
N+1 host reserve	49 + 1 = 50 hosts	98 + 1 = 99 hosts
Capacity after one host failure	49 × 31.2 = 1,528.8 cores	98 × 31.2 = 3,057.6 cores

The calculation assumes that peak CPU demand grows proportionally with the stated workload multiplier and that 48-core hosts are retained. It is a conservative host-equivalent estimate; real sizing must be confirmed by load testing because CPU demand may not scale linearly.

Memory planning: Task 3 assumed average memory demand of 768 GB because the assignment did not provide memory utilization. If that baseline is multiplied by five, the estimated demand is 3,840 GB. Adding 25% planning headroom gives 4,800 GB. At an 80% usable-memory planning limit, each 512 GB host contributes 409.6 GB; 4,800 ÷ 409.6 = 11.72, therefore 12 active hosts plus one N+1 host = 13 hosts. CPU, not memory, is the limiting factor under these assumptions. For the 10× stress case, the same method gives 9,600 GB with headroom, requiring 24 active hosts plus one reserve host = 25 hosts.

Storage planning: the existing 40 TB is total installed local capacity, not measured used data. If the university provisioned the full 40 TB and data capacity grew linearly by five times, the gross planning figure would be 200 TB. With 20% free capacity, approximately 250 TB of logical provisioned capacity would be required. This is an upper-bound planning illustration, not a measured data-growth forecast; actual sizing must use used capacity, retention, replicas and backup copies.

Network planning: the assignment does not state current network throughput, so an absolute bandwidth value cannot be calculated reliably. Measure the current peak throughput B and initially plan for at least 5B aggregate peak capacity (or 10B in the stress case), distributed across redundant links. Validate with concurrent-user, download and API load tests.

11.4 Is the Original Eight-Host Design Sufficient?

No. The eight-host proposal in Task 3 was an initial design for the original workload based on raw host capacity and one-host-failure arithmetic. At a 65% planning target, seven surviving 48-core hosts provide 7 × 48 × 0.65 = 218.4 planning CPU cores, which is below the original estimated 304-core peak. This shows that the earlier peak-fit calculation did not preserve the 65% target at peak. The cluster therefore requires a measured capacity review even before the Day-4 expansion.

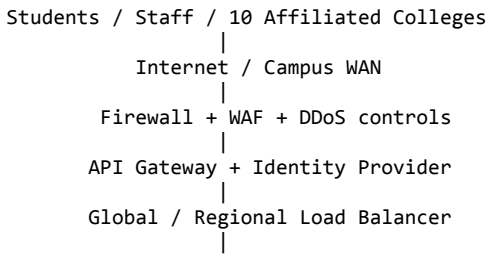
For the five-times peak, approximately 50 host-equivalents are required if all capacity is purchased on-premises under the stated 65% utilization assumption. Procuring all 50 immediately may exceed the limited budget and leave capacity idle outside peak periods. A hybrid model is more practical: retain a suitably validated on-premises base cluster for steady demand and burst stateless web/API/container workloads into a trusted cloud environment during predictable peaks. The cloud burst capacity must be load-tested, secured and budget-controlled.

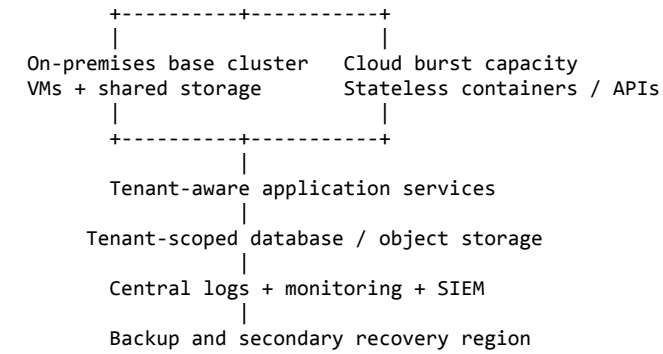
11.5 Meaning of 99.9% Availability:

99.9% availability permits approximately 0.1% downtime. Over a 365-day year: 365 × 24 × 0.001 = 8.76 hours of downtime per year. Over a 30-day month, this is approximately 43.2 minutes. Planned and unplanned downtime must be counted according to the university's approved service-level measurement method.

- Deploy critical application instances across separate failure domains or availability zones where supported.
- Use redundant load balancers, firewalls, switches, power paths and storage controllers.
- Configure health checks and automatic failover; test host and application recovery.
- Maintain independent backups and a secondary recovery location; replication alone is not sufficient.
- Monitor service-level indicators continuously and maintain incident response and change-control procedures.
- Conduct periodic failover and disaster-recovery exercises and measure actual recovery times.

11.6 Revised Architecture and Service Model





The recommended decision is MODIFY, rather than retain the original design unchanged or replace everything. Keep the Type-1 hypervisor foundation and VM placement for legacy/database workloads, but add a hybrid cloud burst layer for stateless services, an API gateway, container orchestration, stronger tenant controls and multi-site recovery.

Design area	Original approach	Modified approach
Compute	Eight on-premises hosts as initial baseline.	Validated on-premises base capacity plus autoscaled cloud capacity for peaks; size from 5× demand tests.
Application platform	VMs for core systems and selected containers.	Keep core VMs; use orchestrated containers for stateless web/API and learning-platform components.
Service models	Primarily on-premises IaaS-style infrastructure.	Hybrid IaaS plus managed PaaS services where they reduce operational burden; SaaS only for suitable commodity functions.
Networking	Department VLANs and virtual switches.	Add tenant-specific network policies, API gateway controls, private connectivity and deny-by-default inter-tenant rules.
Storage and data	Shared storage, snapshots and backup.	Tenant-scoped storage, encryption, lifecycle/quotas, replicated critical data and independent backups.
Availability	Host-level HA and planned DR.	Multi-failure-domain deployment, redundant edge components, tested failover and secondary-region recovery.

11.7 Multi-Tenant Isolation and Security:

- Identity: assign each college a unique tenant identifier and use federated identity or centrally managed accounts with MFA for privileged users.
- Authorization: enforce role-based and tenant-scoped access at the API and application layers; verify tenant ownership on every request.
- Network: separate tenant workloads using virtual networks, VLANs or overlays and firewall policies; deny cross-tenant traffic by default.
- Data: use tenant-aware database schemas or separate databases where risk and compliance require stronger isolation; apply encryption and controlled key access.
- Compute: apply per-tenant CPU, memory, storage and request quotas to prevent one college from exhausting shared resources.
- Audit: log authentication, data access, administrative actions and policy changes; retain logs under an approved retention policy.

11.8 Cost and Operational Trade-Offs:

A fully on-premises 50-host solution for the 5× scenario provides direct control but requires substantial capital expenditure, power, cooling, space and support. A fully cloud-hosted solution reduces hardware procurement but may introduce recurring compute, storage, data-transfer and managed-service charges. The hybrid model balances a stable base workload with temporary burst capacity, but increases network, identity, monitoring and operational complexity.

Option	Advantages	Disadvantages / controls
Expand entirely on-premises	Direct control, predictable hardware ownership and local data placement.	High upfront cost and idle capacity outside peak periods; procurement and installation take time.
Move entirely to public cloud	Elastic capacity and managed services; reduced physical infrastructure responsibility.	Recurring usage and data-transfer costs, governance and vendor lock-in; budgets and exit plans required.
Hybrid model (recommended)	Retains control of core systems while bursting stateless workloads during peaks.	Requires secure connectivity, consistent identity, cost monitoring and skills across both environments.

For the 10×/40-college stress case, the CPU host-equivalent calculation rises to approximately 99 hosts including one reserve. This confirms that the same hybrid strategy would need substantially greater cloud burst capacity, stricter tenant quotas and a new cost approval. It should not be treated as covered by the 5×/10-college design without additional testing.

11.9 Final Decision and Evidence-Based Defence:

Final decision: MODIFY the original design. The existing eight-host proposal is not sufficient for the expanded five-times peak workload and does not maintain the 65% CPU planning target at the original estimated peak. The modified design retains the hypervisor cluster for core workloads but adds cloud burst capacity, container orchestration, an API gateway, multi-tenant isolation, redundant service components and a tested secondary recovery region.

The decision is supported by the 1,520-core 5× peak estimate, 50-host all-on-premises equivalent including N+1 reserve, memory planning of approximately 13 hosts including reserve, and the 99.9% downtime allowance of 8.76 hours per year. Storage and network figures require measured baseline usage and throughput before procurement. The 10×/40-college stress case is separately identified because it appears in the repeated assignment rubric.

Before final deployment, the university should run representative load tests, confirm application compatibility, validate tenant isolation, obtain vendor quotations, calculate cloud burst costs and conduct a failover/DR exercise. These tests are necessary to turn the proposed design into verified operational evidence.

Conclusion: A modified hybrid architecture offers a more practical balance of scalability, availability, security and cost than retaining the original fixed cluster unchanged. Capacity must be purchased and cloud burst policies configured only after measured tests and an approved budget.

REFERENCES

1. National Institute of Standards and Technology (NIST). SP 800-125, Guide to Security for Full Virtualization Technologies. https://csrc.nist.gov/pubs/sp/800/125/final
2. National Institute of Standards and Technology (NIST). SP 800-144, Guidelines on Security and Privacy in Public Cloud Computing. https://csrc.nist.gov/pubs/sp/800/144/final
3. Docker Documentation. Docker Engine and container resource constraints. https://docs.docker.com/
4. Kubernetes Documentation. Workload management, scaling and resource management. https://kubernetes.io/docs/
5. Microsoft Learn. Hyper-V overview and virtualization concepts. https://learn.microsoft.com/windows-server/virtualization/hyper-v/
6. VMware. vSphere documentation: resource management, availability and vMotion concepts. https://docs.vmware.com/
7. Amazon Web Services. Well-Architected Framework: reliability, security, performance efficiency and cost optimization. https://docs.aws.amazon.com/wellarchitected/latest/framework/
8. Google Cloud. Architecture Framework: reliability, security, operational excellence and cost optimization. https://cloud.google.com/architecture/framework